```
> restart;
  with(Physics):with(MTM):with(LinearAlgebra):with(Student):
  # First, you have to be sure that you are using the latest version
  of Physics Package, otherwise some commands may not work properly.
  In this work, we used the update Physics61.3 availabe on
  http://www.maplesoft.com/products/maple/features/physicsresearch.
  aspx
  Physics:-Version();
```

"/Library/Frameworks/Maple.framework/Versions/17/lib/Physics.mla", (1)

2014, March 19, 3:0 hours

```
> Setup(op = S,E,C,P);
```

** Partial match of 'op' against keyword 'quantumoperators'*

** Partial or misspelled keyword matches more than one possible keyword. Please select the correct one from below and try again.*

default, metric, redo, keywords, levicivita, readusersetup, realobjects, redefinesum, vectordisplay, vectorpostfix, deletesavedsetup, hermitianoperators, geometricdifferentiation (2)

```
> # Notation S = "substrate fock space" Sp = Creation Operator for
  the Substract, Sm = Annihilation operator for the Substrate
> # Creating a set of creation operators and ajusting for the
  suitable phase
  N:=1;
  for i from 1 to N do
  Sp[i]:= Creation(S,i,phaseconvention= proc(n) 1 end proc,notation =
  explicit);
  # Same thing for the Annihilation operator
  Sm[i]:=Annihilation(S,i,phaseconvention= proc(n) n end
  proc,notation = explicit);
  end do;
```

$$N := 1$$

$$Sp_1 := a+_{S_1}$$

$$Sm_1 := a-_{S_1} \quad (3)$$

```
> # Notation E = "Enzime fock space" Ep = Creation Operator for the
  Enzime, Em = Annihilation operator for Enzime
> # Creating a set of creation operators and ajusting for the
  suitable phase
  N:=1;
  for i from 1 to N do
  Ep[i]:= Creation(E,i,phaseconvention= proc(n) 1 end proc,notation =
  explicit);
  # Same thing for the Annihilation operator
  Em[i]:=Annihilation(E,i,phaseconvention= proc(n) n end
  proc,notation = explicit);
  end do;
```

$$N := 1$$

$$Ep_1 := a+_{E_1}$$

$$Em_1 := a-_{E_1} \quad (4)$$

```
> # Notation C = "Complex fock space" Cp = Creation Operator for
  Complex, Cm = Annihilation operator for Complex
> # Creating a set of creation operators and ajusting for the
  suitable phase
  N:=1;
  for i from 1 to N do
  Cp[i]:= Creation(C,i,phaseconvention= proc(n) 1 end proc,notation =
  explicit);
  # Same thing for the Annihilation operator
  Cm[i]:=Annihilation(C,i,phaseconvention= proc(n) n end
  proc,notation = explicit);
  end do;
```

$$N := 1$$

$$Cp_1 := a+_{C_1}$$

$$Cm_1 := a-_{C_1} \tag{5}$$

```
> # Notation P = "Products fock space" Ep = Creation Operator for
  Products, Em = Annihilation operator for Products
> # Creating a set of creation operators and ajusting for the
  suitable phase
  N:=1;
  for i from 1 to N do
  Pp[i]:= Creation(P,i,phaseconvention= proc(n) 1 end proc,notation =
  explicit);
  # Same thing for the Annihilation operator
  Pm[i]:=Annihilation(P,i,phaseconvention= proc(n) n end
  proc,notation = explicit);
  end do;
```

$$N := 1$$

$$Pp_1 := a+_{P_1}$$

$$Pm_1 := a-_{P_1} \tag{6}$$

```
> #Reactions
> # First reaction (p)
  # S + E -> C (kp1)
  # First reaction (m)
  # C -> S + E (km1)
  # Seccond reaction
  # C -> E + P (k2)
> H1:=proc(kp1) -Expand(kp1*(Cp[1]-Ep[1]*Sp[1])*Em[1]*Sm[1]); end
  proc;
  H2:=proc(km1) -Expand(km1*(Sp[1]*Ep[1]-Cp[1])*Cm[1]); end proc;
  H3:=proc(k2) -Expand(k2*(Pp[1]*Ep[1]-Cp[1])*Cm[1]); end proc;
```

$$H1 := \mathbf{proc}(kp1)$$

$$Physics:\text{-}Expand(kp1 * (Cp[1] - Ep[1] * Sp[1]) * Em[1] * Sm[1]) * (-1)$$

end proc

$$H2 := \mathbf{proc}(km1)$$

$$Physics:\text{-}Expand(km1 * (Sp[1] * Ep[1] - Cp[1]) * Cm[1]) * (-1)$$

```
end proc
```

$$H3 := \mathbf{proc}(k2)\ Physics:\text{-}Expand(k2 * (Pp[1] * Ep[1] - Cp[1]) * Cm[1]) * (-1)\ \mathbf{end\ proc} \quad (7)$$

```
> H:=proc(kp1,km1,k2) H1(kp1)+H2(km1)+H3(k2); end proc;
```

$$H := \mathbf{proc}(kp1, km1, k2)\ H1(kp1) + H2(km1) + H3(k2)\ \mathbf{end\ proc} \quad (8)$$

```
> H(kp1,km1,k2);
```

$$kp1\, a+_{E_1}\, a+_{S_1}\, a\text{-}_{E_1}\, a\text{-}_{S_1} - kp1\, a+_{C_1}\, a\text{-}_{E_1}\, a\text{-}_{S_1} + km1\, a+_{C_1}\, a\text{-}_{C_1} - km1\, a+_{S_1}\, a+_{E_1}\, a\text{-}_{C_1} \quad (9)$$
$$- k2\, a+_{P_1}\, a+_{E_1}\, a\text{-}_{C_1} + k2\, a+_{C_1}\, a\text{-}_{C_1}$$

```
> #The nonzero contributions for the Hamiltonian are given by Eq.
  [23] of the manuscript and can be found as follows:
> (Bra(C,ci).Bra(S,si).Bra(E,ei).Bra(P,pri)).(H(kp1,km1,k2)).(Ket(P,
  prj).Ket(E,ej).Ket(S,sj).Ket(C,cj));
```

$$\left(\delta_{si,sj}\left((k2+km1)\,cj + kp1\,sj\,ej\right)\delta_{ei,ej}\,\delta_{ci,cj} - \delta_{ei,ej-1}\,\delta_{si,sj-1}\,ej\,sj\,kp1\,\delta_{ci,cj+1} \right. \quad (10)$$
$$\left. - cj\,\delta_{ci,cj-1}\,\delta_{ei,ej+1}\,\delta_{si,sj+1}\,km1\right)\delta_{pri,prj} - cj\,\delta_{ci,cj-1}\,\delta_{ei,ej+1}\,\delta_{pri,prj+1}\,k2\,\delta_{si,sj}$$

```
> #Creating the Basis (Now as a function of sm, em, cm, pm). Here, we
  #suppose some conservations laws to reduce the dimension of fock
  #space. In fact we use that E + C =E0 +C0 and S + C + P =S 0 + C0 +
  #P0. In this case, S0=sm, E0=em C0 = P0 =0.
  Baseset:=proc(sm,em,cm,pm) global B; global ket; global bra;global
  Bdim;
  i:=1;

  for si from 0 to sm
      do
         for ei from 0 to em
             do
                for ci from 0 to cm
                    do
                       for pri from 0 to pm
                           do
                           if ei+ci = em then
                           if si+ci+pri = sm then

                           B[i]:=(si,ei,ci,pri);

                           ket[i]:=Ket(S,si).Ket(E,ei).Ket(C,ci).Ket
  (P,pri);

                           bra[i]:=Bra(P,pri).Bra(C,ci).Bra(E,ei).Bra
  (S,si);
                           print(i,B[i] = ket[i]);
                           i:=i+1;
                           end if;
                           end if;
                           end do;
                    end do;
                end do;
          end do;
      Bdim:=i-1;
```

```
  #for k from 1 to (i-1) do print(i,B[i] = ket[i]); end do;
  end proc;
```

```
Warning, `i` is implicitly declared local to procedure `Baseset`
Warning, `si` is implicitly declared local to procedure `Baseset`
Warning, `ei` is implicitly declared local to procedure `Baseset`
Warning, `ci` is implicitly declared local to procedure `Baseset`
Warning, `pri` is implicitly declared local to procedure `Baseset`
```

```
Baseset := proc(sm, em, cm, pm)                                    (11)
    local i, si, ei, ci, pri;
    global B, ket, bra, Bdim;
    i := 1;
    for si from 0 to sm do
        for ei from 0 to em do
            for ci from 0 to cm do
                for pri from 0 to pm do
                    if ei + ci = em then
                        if si + ci + pri = sm then
                            B[i] := si, ei, ci, pri;
                            ket[i] := Physics:-Ket(S, si).Physics:-Ket(E, ei).Physics:-Ket(C, ci)
                            .Physics:-Ket(P, pri);
                            bra[i] := Physics:-Bra(P, pri).Physics:-Bra(C, ci).Physics:-
                            Bra(E, ei).Physics:-Bra(S, si);
                            print(i, B[i] = ket[i]);
                            i := i + 1
                        end if
                    end if
                end do
            end do
        end do
    end do;
    Bdim := i - 1
end proc
```

```
> # For instance, the baseset choosen in the manuscript is given by:
  Baseset(1,1,1,1);
```

$$1,\ (0,0,1,0) = |S_0\rangle\,|E_0\rangle\,|C_1\rangle\,|P_0\rangle$$

$$2,\ (0,1,0,1) = |S_0\rangle\,|E_1\rangle\,|C_0\rangle\,|P_1\rangle$$

$$3,\ (1,1,0,0) = |S_1\rangle\,|E_1\rangle\,|C_0\rangle\,|P_0\rangle$$

(12)